

学号: _____ 姓名: _____ 分数: _____

1-热身题[11 分]

首先，熟悉一下决策树的相关知识。下面的数据集 D 包含 8 个样本，每个样本有 3 个属性 (A, B, C) ，和一个标签 Y 。

A	B	C	Y
1	2	0	1
0	1	0	0
0	0	1	0
0	2	0	1
1	1	0	1
1	0	1	0
1	2	1	0
1	1	0	1

使用上述数据回答以下问题并注意：

- 在计算过程中，所有的计算都不能四舍五入！在你完成所有的计算后，将你的解进行四舍五入，作为最终的答案。
 - 数字解应包括 4 位精度(例如 0.1234)，除非另有特殊说明。
 - 注意，在整个作业中，都使用**树的叶子不算作节点**的惯例，因此叶节点不包括在深度和分裂数的计算中。(例如，对于只根据一个属性的值来对数据进行分类的树的深度为 1，并进行了 1 次分裂)
 - 注意，数据集包含重复的行；将其中的每一行都视为单独的示例，不要删除重复的行。
1. (1 分) Y 的熵是多少，也就是 $H(Y)$ 的值是多少？在这个以及后面的问题中，在计算中请使用以 2 为底的对数。(请将你的答案四舍五入到小数点后第四位的数字，例如 0.1234)
 2. (1 分) Y 和 A 的互信息是多少，也即 $I(Y; A)$ 的值是多少？(请将你的答案四舍五入到小数点后第四位的数字，例如 0.1234)
 3. (1 分) Y 和 B 的互信息是多少，也即 $I(Y; B)$ 的值是多少？(请将你的答案四舍五入到小数点后第四位的数字，例如 0.1234)
 4. (1 分) Y 和 C 的互信息是多少，也即 $I(Y; C)$ 的值是多少？(请将你的答案四舍五入到小数点后第四位的数字，例如 0.1234)
 5. (1 分)单选:根据上面给出的数据集，如果以互信息作为分裂准则，下面哪个属性(A, B 或 C)是决定树算法的第一个选择分支？
A.A B.B C.C
 6. (1 分)单选:根据上面给出的数据集，在进行第一次分裂后，如果分裂标准为互信息，算法将选择哪个属性进行下一步分支？(注意，这个问题假设了第二个属性只有一种选择)
A.A B.B C.C

7. (1 分)如果同样的算法继续进行，直到树完全将每个样本进行分类，那么树的深度是多少？
8. (4 分)画出你完整的决策树。标记出树在其属性上分裂的非叶子节点(例如 B)，标记出具有属性值的边缘(例如 1 或 0)，以及具有分类决策的叶子节点(例如 $Y = 0$)。可以手绘你的决策树。

2-伪代码[6 分]

1. 在编程作业中，你将需要实现三个任务:一，在任意训练集上训练决策树，二，用训练后的决策树对给定的输入数据集进行预测，三，根据数据集的真实标签评估程序的预测。下面，你将为函数 `predict (node, example)` 编写伪代码，该函数在给定一个 `node` 类型的节点(表示训练树的根)的情况下预测样本的标签。你需要使用代码中提供给你的 `Node` 类，并递归地解答上面的问题。

```
class Node:
    def __init__(self, attr, v):
        self.attribute = attr
        self.left = None
        self.right = None
        self.vote = v
# (a) the left and right children of a node are denoted as
#     node.left and node.right respectively, each is of type Node
# (b) the attribute for a node is denoted as node.attribute and has
#     type str
# (c) if the node is a leaf, then node.vote of type str holds the
#     prediction from the majority vote; if node is an internal
#     node, then node.vote has value None
# (d) assume all attributes have values 0 and 1 only; further
#     assume that the left child corresponds to an attribute value
#     of 1, and the right child to a value of 0
def predict(node, example):
    # example is a dictionary which holds the attributes and the
    # values of the attribute (ex. example['X'] = 0)
```

(a)(3 分)写出 `predict(node, example)` 的最基本情况。将代码限制在 10 行以内。

(b)(3 分)写出 `predict(node, example)` 的递归解法。将代码限制在 10 行以内。

3-实证问题(13 分)

下面的问题应该在你完成编程部分时来回答。

1. (4 分)在心脏病数据集和教育数据集上用四个不同的 max-depth 值{0,1,2,4}来训练和测试你的决策树。填写下面的表格，并说说你发现什么。具有最大深度为 0 的决策树是简单的多数投票分类器;最大深度为 1 的决策树称为决策树桩。(请将每个答案四舍五入至小数点后第四位，例如 0.1234)

Dataset	Max-Depth	Train Error	Test Error
heart	0		
heart	1		
heart	2		
heart	4		
education	0		
education	1		
education	2		
education	4		

2. (3 分)对于心脏病数据集，创建一个由计算机程序生成的图，其中，y轴表示误差，x轴表示树的深度。在单个图上，需要同时显示训练误差和测试误差，明确标注哪个是训练误差，哪个是测试误差。也就是说，对于 max-depth(0,1,2, ...)的每个可能值(直到数据集中的属性数量)，你应该训练决策树并报告你的决策树预测的训练/测试错误。你的文件名应为 heart.png。
3. (2 分)单选:假设你的老师要求你进行一些模型选择实验，然后报告你的结果。你选择了决策树模型中具有最低测试误差的最大深度 (max-depth) 的模型，然后将此模型在测试集上的错误率作为分类器在保留测试数据上的性能。这是一个好的选择吗？
 - A. 是的，因为我们使用测试集是为了选择最好的模型。
 - B. 不，因为我们应该用训练集来优化最大深度，而不是用测试集。
 - C. 是的，因为我们没有使用训练集来调整超参数。
 - D. 不，因为我们应该使用验证集来优化最大深度，而不是用测试集
4. (2 点)单选:在这个任务中，我们使用最大深度作为我们的停止继续分裂的准则，并作为防止过拟合的机制。我们可以在最佳属性的互信息低于某个阈值时停止拆分节点。这个阈值是另一个超参数。从理论上讲，增加这个阈值会如何影响学习决策树的节点数量和深度？
 - A. 这个阈值越高，决策树的深度越小，包含的节点越少。
 - B. 这个阈值越高，决策树的深度越小，包含的节点越多。
 - C. 这个阈值越高，决策树的深度越大，包含的节点越少。
 - D. 这个阈值越高，决策树的深度越大，包含的节点越多。
 - E. 决策树中节点的深度和数量不会随着阈值的增加而变化。
5. (2 分) 在心脏数据集，打印(禁止手写)最大深度为 3 的决策树。关于如何打印树的说明可以在下面的 4.7 节中找到。

4 编程(70 分)

本次作业的目标是实现一个完全从零开始的二分类器——具体来说就是一个决策树学习器。此外，我们将要求你在两个任务(预测患者是否患有心脏病和预测高中生的最终成绩)上运行一些端到端实验并报告你的实验结果。你需要编写两个程序:inspection.py(第 4.2 节)和 decision_tree.py(第 4.3 节)。

4.1 任务和数据集

下载数据。

data 目录下包含 decision_tree.py 文件，该文件包含一些作业的初始代码，建议你在初始代码的基础上编写自己的代码，完成本次作业。

data 目录中包含三个数据集。每个样本都包含属性和标签，并且已经分为训练数据和测试数据。每个 tsv 文件的第一行包含各个属性的名称，而最后一列表示类别标签。

心脏病数据集:

训练数据在 heart_train.tsv 中，测试数据在 heart_test.tsv 中。第一个任务是根据现有的病人信息，预测病人是否被诊断为患有心脏病。各属性的说明如下:

属性名称	含义
sex	性别: 患者的性别——1 表示患者是男性, 0 表示患者是女性。
chest_pain	胸痛: 如果患者胸痛, 则为 1, 否则为 0。
high_blood_sugar	高血糖: 如果患者有高血糖(空腹 > 120 mg/dl), 则为 1, 否则为 0
abnormal_ecg	心电图异常: 运动诱发心绞痛为 1, 其他为 0。
angina	心绞痛是一种严重的胸痛
flat_ST	ST 段是否为平: 如果患者的 ST 段(心电图的一部分)在运动时是平的, 则为 1, 如果有斜率则为 0
fluoroscopy	荧光透视法: 如果医生使用透视检查, 则为 1, 否则为 0。透视是一种用来观察心脏血流的成像技术
thalassemia	地中海贫血: 如果患者已知患有地中海贫血, 则为 1, 否则为 0。地中海贫血是一种血液疾病, 可能会损害患者红细胞的携氧能力
heart_disease	心脏病: 如果患者被诊断患有心脏病, 则为 1, 否则为 0。这是你需要预测的类标签。

教育数据集

训练数据在 education_train.tsv 中，测试数据在 education_test.tsv 中。

第二个任务是预测高中生的最终成绩。 这些属性分别是 5 个选择题 M1 到 M5, 4 个编程作业 P1 到 P4 和期末考试 F 的得分。最终成绩的取值为 1 则表示该学生得到 A; 最终的成绩取值为 0 则表示该学生没有得到 A。

小型心脏病数据集

训练数据在 small_train.tsv 中，测试数据在 small_test.tsv 中。

小型心脏病数据集是一个用于演示版本的心脏病数据集，只有胸痛(chest_pain)和地中海贫血(thalassemia)这两个属性。对于这个小型心脏病数据集，data 文件夹还包含了一个最大深度为 3 的决策树的预测(参见 small_3_train.txt, small_3_test.txt, small_3_metrics.txt)。你可以根据这些结果来检查自己的输出，看看你的实现是否正确。

注意：本次作业中的所有数据集的所有属性都只有两个类别(即每个节点最多有两个子节点)。

4.2 程序 1: 检查数据(5 分)

编写一个程序 inspection.py 来计算根处的标签熵(即未进行任何分裂之前的标签熵)，使用多数投票分类器 MajorityVoteClassifier(选择具有最多示例的标签)进行分类，并计算错误率(分类错误的样本数占总样本数的百分比)。你不需要查看任何属性的值来进行这些计算;知道每个例子的标签即可。计算熵时应使用以 2 为底的对数来计算。

命令行参数

你需要使用以下命令运行程序：

```
$ python inspection.py <input> <output>
```

你的程序应该接受两个命令行参数:一个输入文件和一个输出文件。应该是 tsv 文件作为输入文件(4.1 节中描述的格式)，计算上述描述的指标量，并将其写入输出文件，输出文件应包含如下内容：

```
entropy: <entropy value>
error: <error value>
```

例子

比如你以 small_train.tsv 作为输入文件并将结果写入 small_inspect.txt，那么你需要运行如下命令：

```
$ python inspection.py small_train.tsv small_inspect.txt
```

之后，你的输出文件 small_inspect.txt 应该包含以下内容：

```
entropy: 1.000000
error: 0.500000
```

你的程序应正确地计算熵和误差。不需要对你的结果进行四舍五入！

请在 data 目录中的每个数据集上运行你的程序，并获得对应的多数投票分类器的错误率。

4.3 程序 2:决策树学习器(65 分)

在 `decision_tree.py` 中, 实现一个决策树学习器。该文件应该学习具有指定最大深度的决策树, 以指定格式打印对应的决策树, 预测训练集和测试集中样本的标签, 并计算预测的误差。

你需要满足以下要求:

- 使用互信息来确定要分割的属性。
- 确保正确计算互信息的权重。
对于属性 X 的互信息计算:
$$I(Y; X) = H(Y) - H(Y | X) = H(Y) - P(X = 0)H(Y | X = 0) - P(X = 1)H(Y | X = 1)$$
- 只在互信息 > 0 时才对一个属性进行分裂。
- 不要使树的深度超出命令行上指定的最大深度。例如, 当最大深度为 3 时, 只在互信息 > 0 且节点当前深度 < 3 时才对节点进行拆分。
- 使用每个叶子上标签类别最多的来做为最终的分类类别。如果票数相等, 则选择较高的标签(即如果标签 0 和 1 的数量相同, 则选择标签 1 作为最终的分类结果输出)。
- 对于相互信息, 不同的列可能具有相等的值。在这种情况下, 你应该选择分割第一列(例如, 如果列 0 和列 4 具有相同的相互信息, 则使用列 0)。
- 不要将数据集的任何方面硬编码到代码中。

这里有一些提示可以帮助你开始:

- 编写辅助函数来计算熵和互信息。
- 最好将决策树视为节点的集合, 其中的节点要么是叶节点(做出最终决策的地方), 要么是内部节点(根据属性进行划分的地方)。可以设计一个函数来训练单个节点(即深度 0 树), 然后递归地调用该函数来创建子树。
- 在递归中, 跟踪当前树的深度, 这样你就可以避免使树超过 `max-depth`。
- 实现一个函数, 该函数以训练后的决策树和要预测的数据作为输入, 并生成对应的预测标签。你可以编写一个单独的函数来计算预测标签相对于给定的真实标签的误差。
- 注意处理如果指定的最大深度大于属性总数的情况。注意处理 `max-depth` 为零的情况(即多数投票分类器)。

4.4 命令行参数

使用以下命令运行程序:

```
$ python decision_tree.py [args...]
```

[args]是六个命令行参数的占位符:<train input> <test input> <max depth> <train out> <test out> <metrics out>。这些参数详细描述如下:

1. <train input>:训练输入的 tsv 文件的路径(参见 4.1 节)
2. <test input>:测试输入的 tsv 文件的路径(参见 4.1 节)
3. <max depth>:构建树的最大深度
4. <train out>:输出 txt 文件的路径, 即训练数据的预测写入该文件(参见 4.5 节)
5. <test out>:输出 txt 文件的路径, 即测试数据的预测写入该文件(参见 4.5 节)
6. <metrics out>: output.txt 文件的路径, 其中应该写入训练集和测试集的预测误差之类的指标(参见第 4.6 节)

例如，下面的命令行将在心脏病数据集上运行你的程序，并学习一个最大深度为 2 的树。对于训练集的预测结果将写入 heart_2_train.txt, 对于测试集的预测结果将写入 heart_2_test.txt, 相关的评估指标将写入 heart_2_metrics.txt。

```
$ python decision_tree.py heart_train.tsv heart_test.tsv 2 \
    heart_2_train.txt heart_2_test.txt heart_2_metrics.txt
```

下面的例子将运行和上述例子相同的学习设置，只是树的最大深度为 3，并且写入类似命名的输出文件，方便你比较两次运行。

```
$ python decision_tree.py heart_train.tsv heart_test.tsv 3 \
    heart_3_train.txt heart_3_test.txt heart_3_metrics.txt
```

4.5 输出:

标签文件: 你的程序应该输出两个 output.txt 文件，分别对应你的模型对训练数据(<train out>)和测试数据(<test out>)的预测。文件的内容应该包含对应的数据集的每个样本的预测标签，使用'\n'创建新行。下面给出了使用小型心脏病数据集的输出文件的前几行:

```
1
0
1
1
0
0
...
```

4.6 输出:Metrics 文件

生成对应的指标文件，你应该在指标文件中写入训练错误和测试错误。将该文件写入命令行参数<metrics out>指定的路径。你答案应该和参考答案的误差在 0.0001 以内。不需要对你的结果进行四舍五入!指标文件的内容示例如下:

```
error(train): 0.214286
error(test): 0.285714
```

上面的值对应于在小型心脏病数据集的训练集上训练深度为 3 的决策树，并在 small_test.tsv 测试集上进行测试的结果。(注意冒号和数值之间有一个空格。)

4.7 输出:打印树

你需要编写一个函数来打印学习到的决策树。只有当决策树在完全训练之后，函数才应该打印对应的决策树。每一行应该对应于决策树中的一个节点。应该使用深度优先搜索遍历来打印(这里规定先遍历左子树再遍历右子树的 DFS 方式，即你的答案需要与下面的参考具有完全相同的顺序)。打印此节点的父节点的属性和对应于该节点的属性值。还要包括传递给该节点的数据的充分统计信息(即正反例的计数)。根的行应该只包含那些足够的统计信息。深度为 d 的节点，应该以字符串'|'的 d 个副本作为前缀。

下面，我们提供了打印树的格式。你可以直接在终端打印出来，而不用输出打印到文件中。

```
$ python decision_tree.py small_train.tsv small_test.tsv 2 \
small_2_train.txt small_2_test.txt small_2_metrics.txt

[14 0/14 1]
| chest_pain = 0: [4 0/12 1]
| | thalassemia = 0: [3 0/4 1]
| | thalassemia = 1: [1 0/8 1]
| chest_pain = 1: [10 0/2 1]
| | thalassemia = 0: [7 0/0 1]
| | thalassemia = 1: [3 0/2 1]
```

你需要注意树的深度没有达到指定的 `max_depth` 的情况。例如，使用的某个心脏病测试数据集，如果数据集里的所有标签都是同一类，则在 `chest_pain = 0` 下可能没有节点。下图显示了最大深度为 3 的教育数据集。可以用这个例子来检查你的结果。

```
$ python decision_tree.py education_train.tsv education_test.tsv 3 \
edu_3_train.txt edu_3_test.txt edu_3_metrics.txt

[65 0/135 1]
| F = 0: [42 0/16 1]
| | M2 = 0: [27 0/3 1]
| | | M4 = 0: [22 0/0 1]
| | | M4 = 1: [5 0/3 1]
| | M2 = 1: [15 0/13 1]
| | | M4 = 0: [14 0/7 1]
| | | M4 = 1: [1 0/6 1]
| F = 1: [23 0/119 1]
| | M4 = 0: [21 0/63 1]
| | | M2 = 0: [18 0/26 1]

| | | M2 = 1: [3 0/37 1]
| | M4 = 1: [2 0/56 1]
| | | P1 = 0: [2 0/15 1]
| | | P1 = 1: [0 0/41 1]
```

括号中的数字给出了该部分训练数据在此节点上标签为 0 的数量和标签为 1 的数量，你应该回到前面 **3-实证问题** 并回答问题 1-5 部分。

4.8 提交说明

编程部分需要完成的文件:

inspection.py

decision_tree.py

不用提交，实验课上当堂进行结果检查，并登记分数。

书面部分需要提交的文件:

回答所有问题后，请提交 PDF 格式的文件。具体命名规则：HW2_张三_22920202209999.pdf
标清题号，只写解题过程和答案即可，不要带题目！